

使用及开发指南

MSU2 系列可编程 USB 屏幕

日期	版本	描述	作者
2024.10.30	1.00	初稿完成	Einx
2024.11.2	1.10	添加字库文件的生成和烧录说明 添加单色图片普通显示和叠加显示的说明	Einx

本文档的版权和解释权归墨砺工作室所有,仅以 PDF 格式对外发布.

墨砺工作室

MORI ATELIERS

目录

1.适用对象.....	1
2.主控固件更新.....	2
3.闪存固件更新.....	5
4.自定义背景图片.....	6
5.自定义待机动图.....	7
6.注意事项.....	10
7.硬件说明(MSU2).....	11
8.硬件说明(MSU2_MINI).....	13
9.API 接口说明.....	14

1.适用对象

MSU2 系列可编程 USB 屏幕是墨砺工作室基于 CH32X033F8P6 微控制器和 P25D80 闪存芯片进行开发的电脑副屏，本指南适用于 1.54 寸屏幕的 MSU2 和 0.96 寸屏幕的 MSU2_MINI 以及采用相同控制电路的同类型产品。

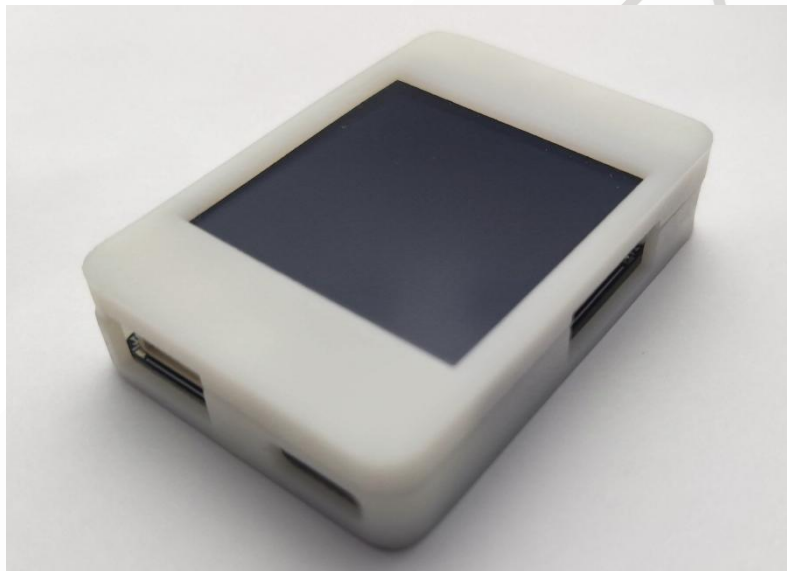


图 1.0 MSU2 外观图

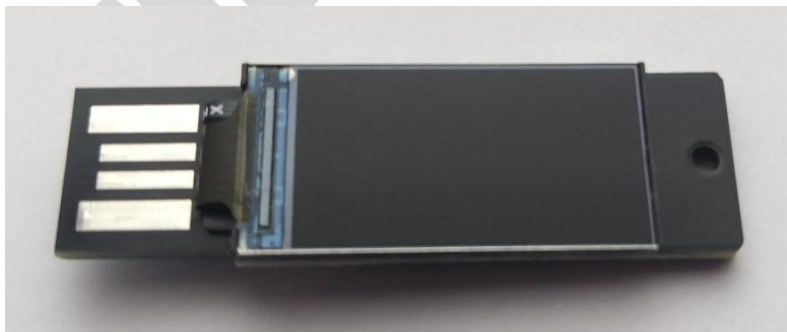


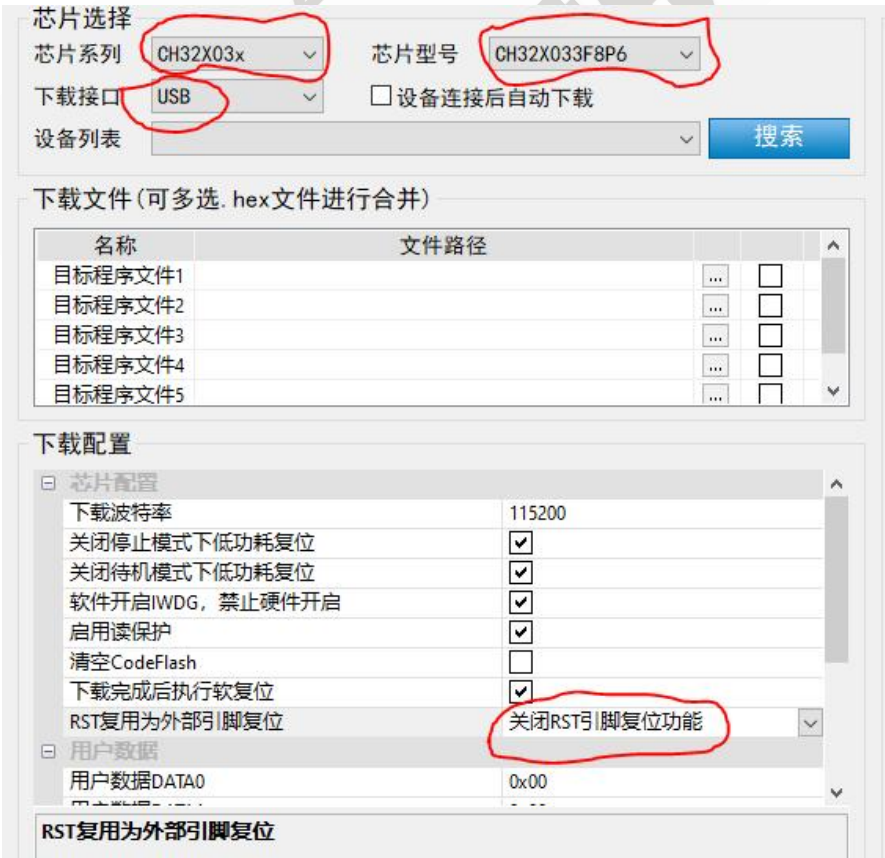
图 1.1 MSU2_MINI 外观图

2.主控固件更新

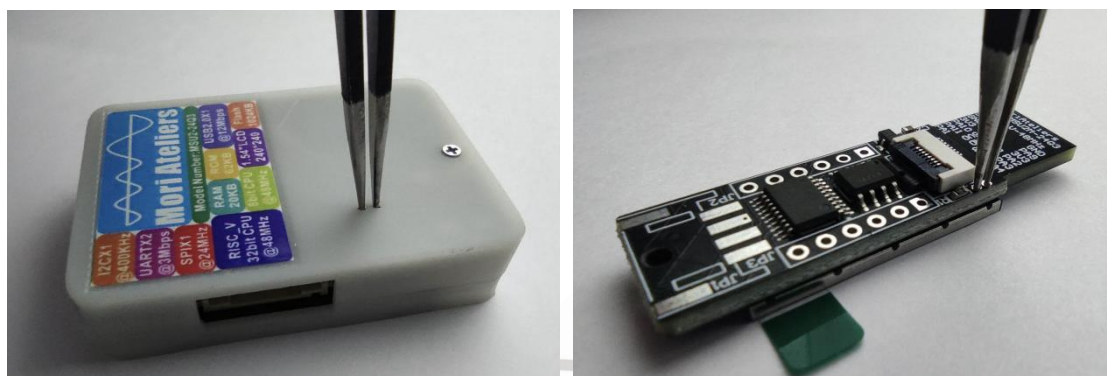
主控采用南京沁恒微电子的 CH32X033F8P6 微控制器，更新主控固件需要其官方固件烧录工具“WCHISPTool”，该软件位于资料压缩包“固件烧录工具及驱动”文件夹中的“固件烧录工具”文件夹，如下图所示：



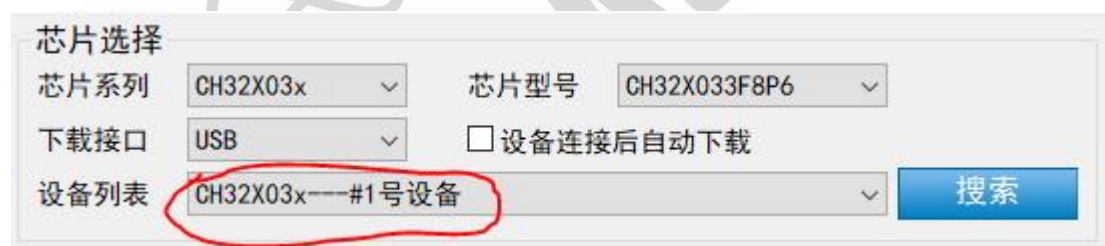
安装完成“WCHISPTool”固件烧录工具之后，在芯片系列选择 CH32X03X，在芯片型号选择 CH32X033F8P6，下载接口选择 USB，并要关闭 RST 引脚复位功能，如下图所示：



主控芯片进入烧录模式需要将芯片第 6 脚“PC17/USB_D+”接到 3.3V 再上电，在 MSU2 和 MSU2_MINI 预留了过孔用于短接 USB_D+ 和 3.3V，推荐使用金属镊子进行短接，如下图所示：



在保持 USB_D+ 和 3.3V 短接的情况下将 MSU2 或 MSU2_MINI 插到电脑的 USB 口，插入 USB 口 1~3 秒后移除金属镊子让 USB_D+ 和 3.3V 断开，此时主控芯片进入 USB 烧录模式，会在软件界面看到设备列表中出现“CH32X03x---#1 号设备”：



然后点击“...”来选择需要烧录的主控固件，并确保后面的框框勾选上，随后点击解除代码保护，再点击下载就会开始烧录固件：



显示下图时说明烧录成功：

```

03:30:56:871>> 开始解除设备代码保护...
03:30:57:085>> 成功!
Device#0 UID:CD-AB-62-6B-77-BC-A6-D3, BTVER:02.60
03:30:59:726>> 待下载BIN文件长度:24224B
03:30:59:837>> [#Dev0]开始下载...
03:30:59:874>> BTVER:02.60
03:30:59:902>> UID:CD-AB-62-6B-77-BC-A6-D3
03:30:59:989>> 擦除中...
03:31:00:019>> 擦除长度: 24K
03:31:00:118>> 完成
03:31:00:153>> 编程中...
03:31:00:521>> 完成
03:31:00:550>> 校验中...
03:31:00:862>> 完成
03:31:00:897>> 成功!
03:31:00:930>> <<<< 本次用时:1.092s
  
```

3.闪存固件更新

MSU2 系列可编程 USB 屏幕使用 P25D80 闪存芯片进行图像、字库等文件的存储，具有 1024KB 的存储空间，10 万次烧写寿命以及在常温下可以保存数据 20 年。当更换闪存芯片或者闪存芯片的数据被破坏时，可以使用“闪存固件更新”功能来将闪存芯片的内容恢复到出厂时烧录的数据，具体操作如下：

- 1.点击“选择闪存固件”后在跳出的弹框中选择“Flahs_Vx.x.bin”文件，“Flahs_Vx.x.bin”即为想要烧录的闪存固件，后续会不断更新；
- 2.选择好需要烧录的文件后点击“烧写”，等待 10 秒左右，直到左下角出现“烧写完成，耗时.....”说明闪存固件更新完成；



4. 自定义背景图片

MSU2 系列可编程 USB 屏幕可以通过上位机软件来修改电子相册里的图像和桌面时钟的背景图像，图像格式支持 JPG\JPEG\BMP\PNG 四种格式，选择好图片后，软件会自动居中裁剪图片来适配屏幕尺寸（1.54 寸屏幕会自动使用 1:1 的长宽比进行裁剪，0.96 寸屏幕会自动采用 2:1 的长宽比进行裁剪），具体操作如下：

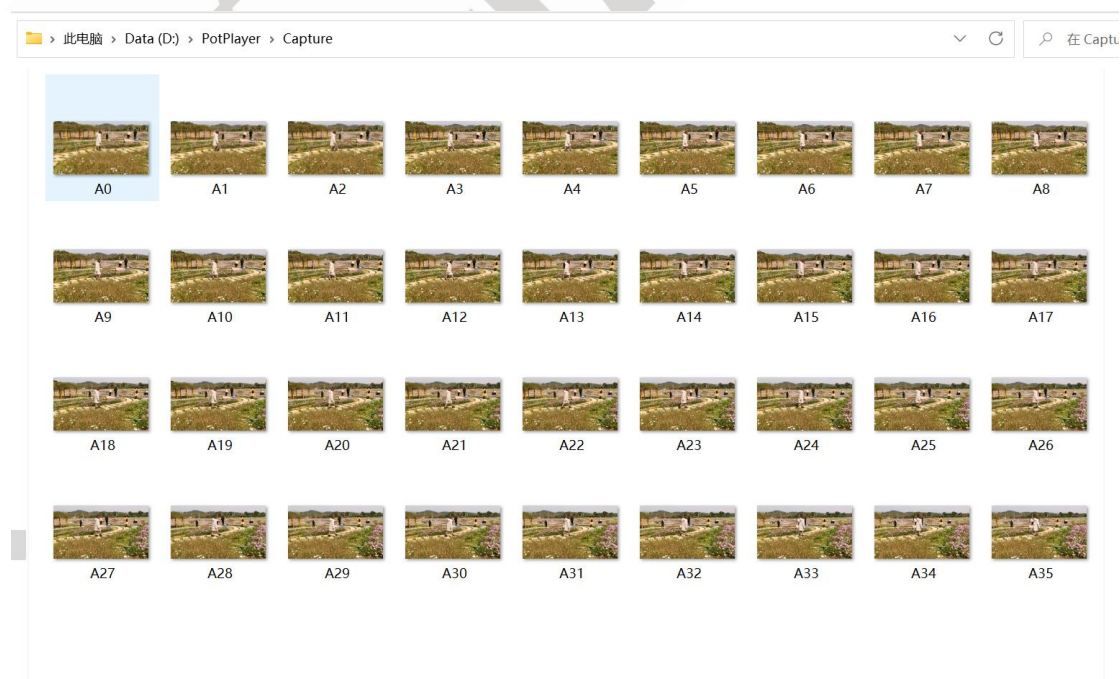
1. 点击“选择背景图像”后在跳出的弹框中选择需要的图像，接着点击“烧写”就可以写入图像，等待 1 秒左右，直到左下角出现“烧写完成，耗时.....”说明图片烧写完成完成；
2. 更换电子相册图像也是采用类似的方法，需要点击“选择相册图像”来选择图像，再点击“烧写”；



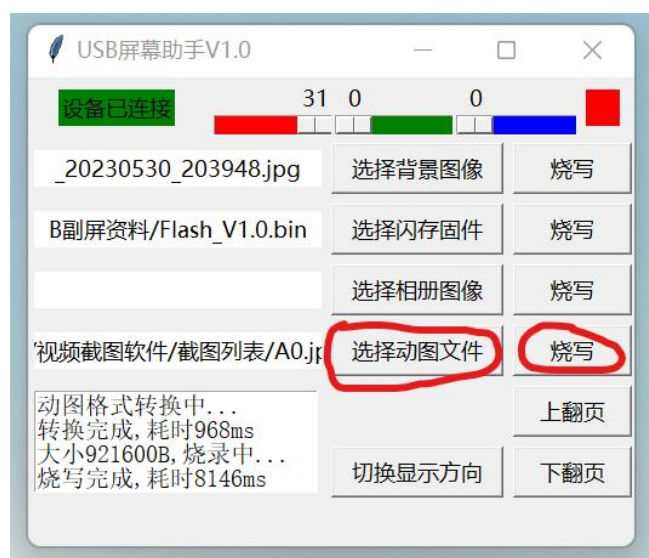
5. 自定待机动画

MSU2 系列可编程 USB 屏幕在离线状态下会自动循环播放 36 张彩色图片，每隔 100ms 切换一次图像来构成时长 3.6S 的彩色动图，这个动图用户可以通过上位机软件自行修改，具体操作如下：

1.预先准备好名称为 A0~A35 的 JPG\JPEG\BMP\PNG 图像，并将其放入同一个文件夹,如下图所示(注意：这 36 张图片的格式需要统一为 JPG\JPEG\BMP\PNG 中的一种，并且名称必须是 A0~A35，缺少图像时会在图像转换过程中卡住；图像过多时，多出来的图像不会被烧录到闪存芯片中；建议图片分辨率不宜大于 2000X1000，否则转换图片格式会耗时过长导致小屏幕断开和上位机软件的连接；软件会自动将图片进行居中的 2:1 裁剪，如果不想居中裁剪的话需要自行使用图像处理软件进行裁剪)：

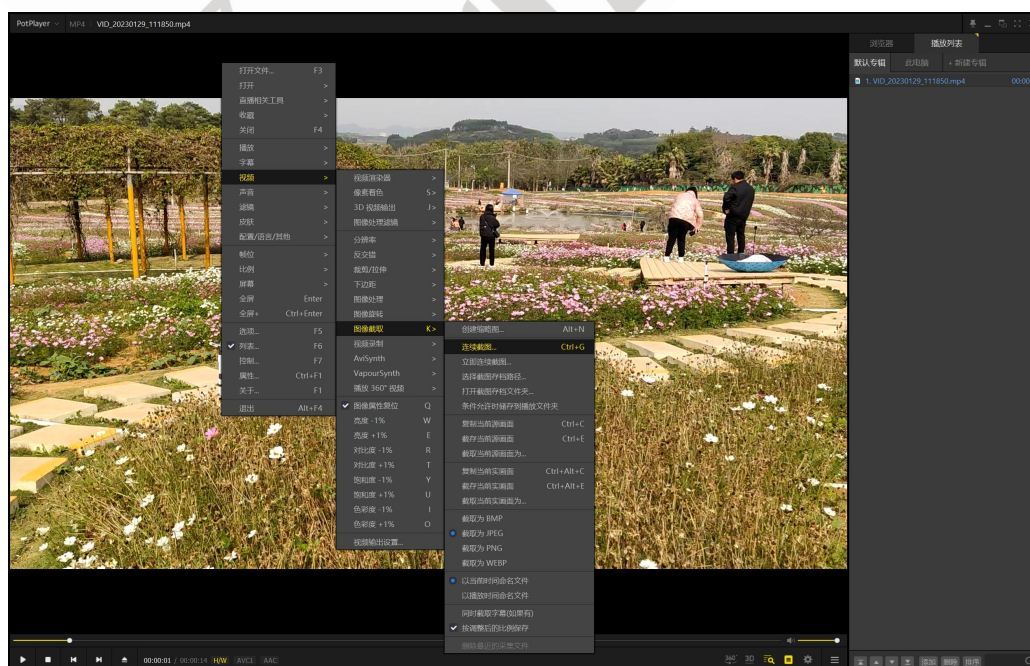


2.准备好图像后，点击“选择动图文件”在跳出来的弹框中定位到准备好的图像文件夹，并选择里面名称为 **A0** 的图像，接着点击“烧写”就可以写入动图，等待 **10** 秒左右，直到左下角出现“烧写完成，耗时.....”说明动图烧写完成完成；



附录：使用 PotPlayer 对视频进行截图生成需要的 A0~A35 图像

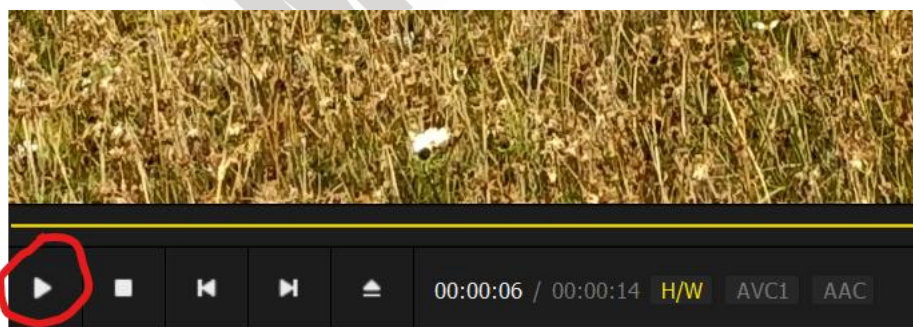
1.使用 PotPlayer 打开视频后，按 Ctrl+G 进入连续截图，或者按下鼠标右键如下图来选择进入连续截图：



2.连续截图页面配置如下（离线状态下主控是每 100ms 切换一次图像，所以截图间隔时间最好设置为 100ms，这样动图播放不会过快或者过慢）：



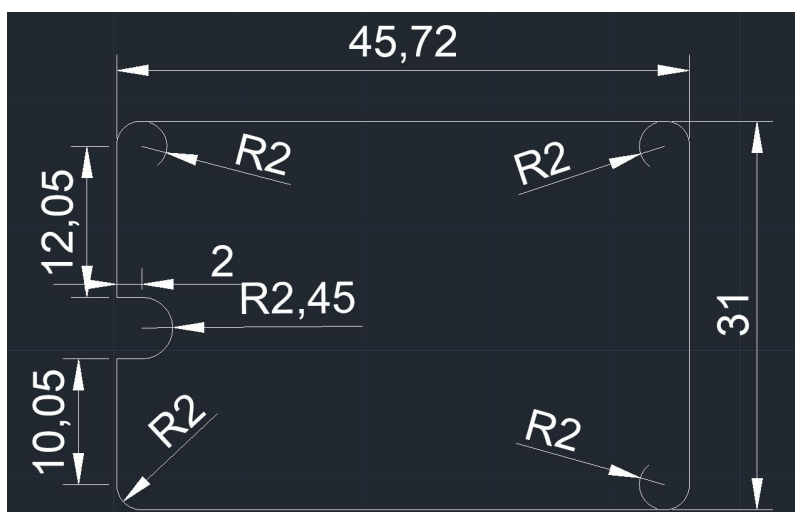
3.点击“开始”后其实还没有真正开始截图，需要按下播放键才开始截图；



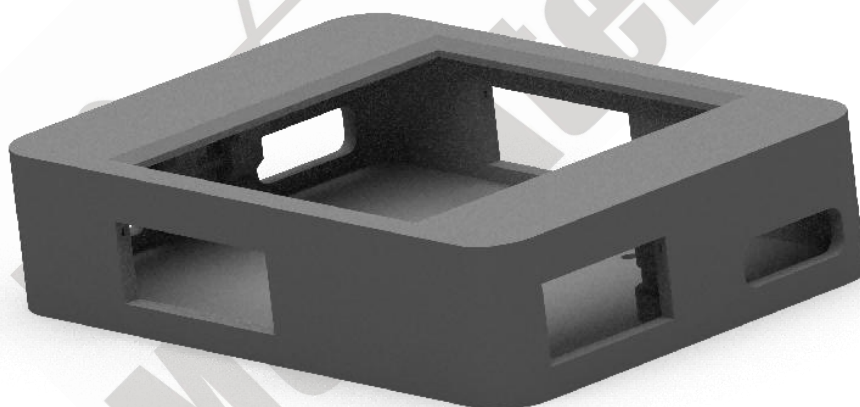
6. 注意事项

1. MSU2 系列可编程 USB 屏幕在 Win10/11 是免驱动的，在 Win7/XP 需要额外的驱动程序；
2. 插入小屏幕后系统并不会自动识别到该屏幕，无法作为扩展屏幕使用；
3. 进行桌面投屏和电脑性能监控需要运行店铺提供的程序；
4. 小屏幕与部分电脑的蓝牙会有冲突，遇到报错时可以尝试关闭蓝牙后再运行软件；

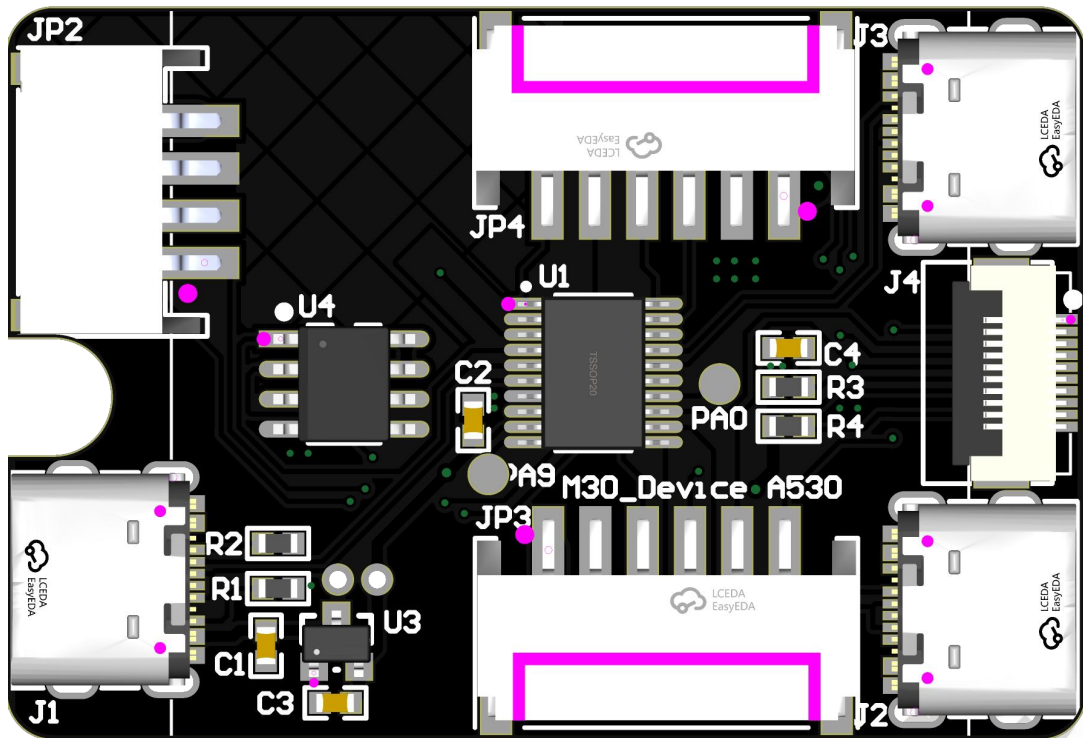
7.硬件说明(MSU2)



PCB 尺寸图
(长 45.72mm,宽:31mm,厚度 1mm)



外壳尺寸图
(长 48.9mm,宽:34.1mm,高 11.2mm)



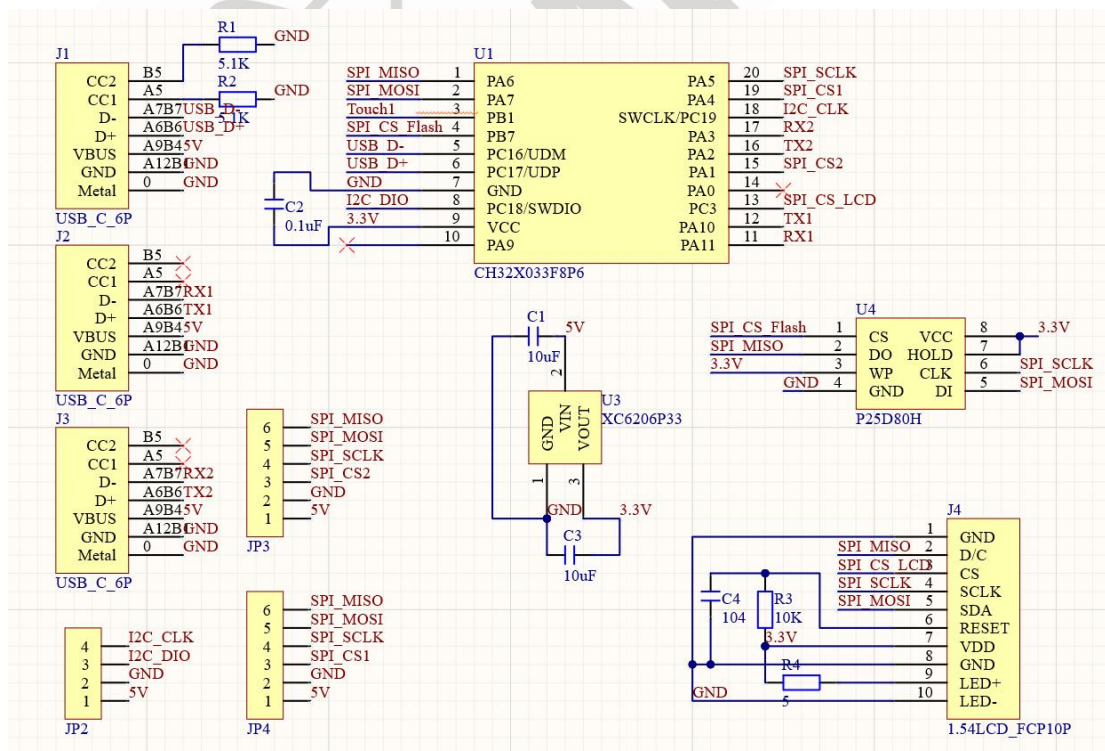
JP2:I2C 接口：用于连接具有 I2C 接口的外部设备

J1:USB 接口：用于与电脑端通信

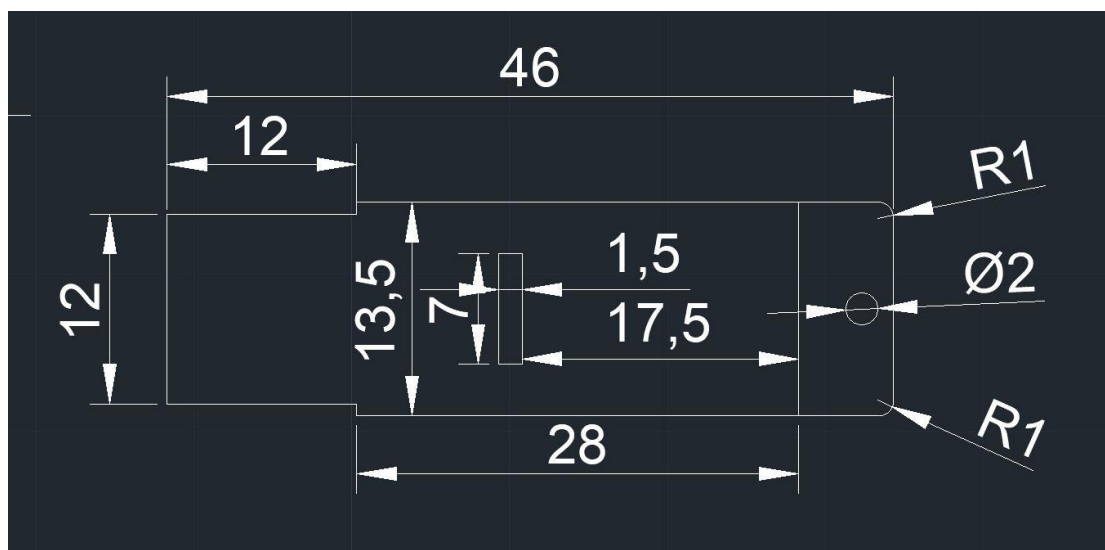
JP3/JP4:SPI 接口：用于连接具有 SPI 接口的外部设备

J2/J3:UART 接口：用于连接具有 UART 接口的外部设备

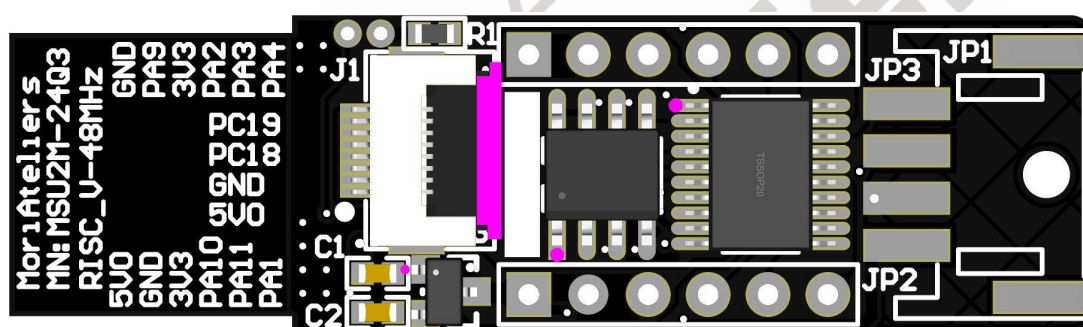
J4:LCD 屏幕接口(1.54 寸屏幕，ST7789)



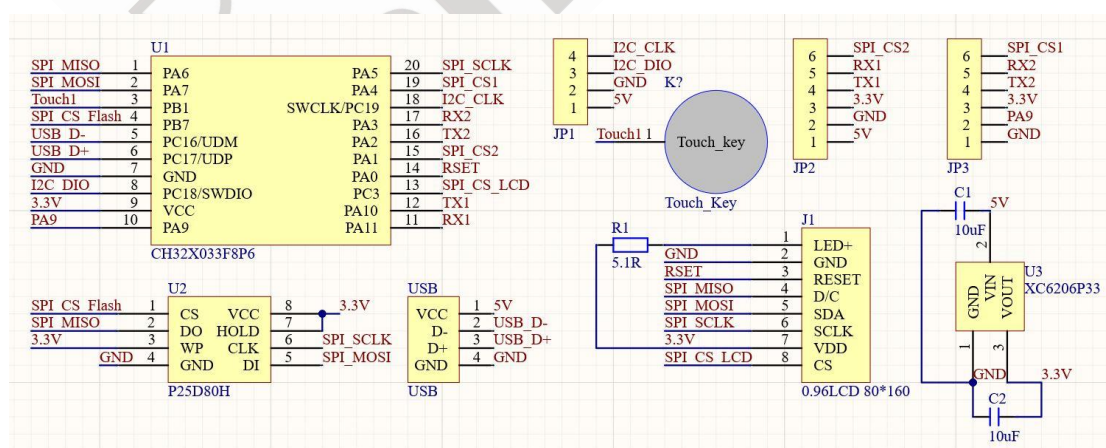
7. 硬件说明(MSU2_MINI)



PCB 尺寸图
(长 46mm, 宽:12mm, 厚度 2mm)



PCB 底部仿真图



电路原理图

USB:USB 接口：用于与电脑端通信

J1:LCD 屏幕接口(0.96 寸屏幕，ST7735S)

JP1:I2C 接口：用于连接具有 I2C 接口的外部设备

JP2/JP3:排针接口：用于连接外部设备

9.API 接口说明

MSU2 系列可编程 USB 屏幕提供了上位机软件的 Python 源码，源码中包含了众多应用程序编程接口(Application Programming Interface)简称 API 接口，通过这些接口函数可以控制 MSU2 所连接的屏幕、闪存、RGB 彩灯、传感器等设备，这些 API 接口大多数是用于操控外部设备，在使用这些接口前应当对这些外部设备有所了解。

1.闪存芯片相关 API 接口

硬件说明:MSU2 系列可编程 USB 屏幕使用的闪存芯片型号为 P25D80, 具有 1024KB 的存储空间，这类闪存芯片以 256B 为一页，可以划分为 4096 页，擦除和写入均要以页为单位进行，而读取是可以随意指定地址和长度的，因此在软件上使用 0~4095 作为页地址，需要值得注意的是擦除是指将 bit 重置为 1，而写入是只能将 bit 由 1 变成 0 而无法将 0 变成 1，即要确保数据的正确写入，应当要在写入之前将指定区域位置给擦除好。

①按页来擦除指定区域的闪存(页地址，页数量)

Erase_Flash_page(add,size)

举例: Erase_Flash_page(20,4)

对第 20、21、22、23 页的闪存进行擦除，总的擦除大小为 $4 \times 256B = 1024B$

2.常用 API 接口说明

①将 bin 格式的图像写入闪存芯片(页地址, bin 格式文件的名称)

`Write_Flash_Photo_fast(Page_add,Photo_name)`

举例: `Write_Flash_Photo_fast(3600,'Demo1')#240*240` 单色图片, 占用 29 个 Page

将 Demo1.bin 文件写入闪存芯片第 3600 页处(注意: 要写入的.bin 文件必须要和当下执行的 python 文件在同一个文件夹目录中; 当前页写不完.bin 文件时会自动写到下一页, 直到.bin 文件写完), Demo1.bin 是 240*240 单色图片, 其大小为 $240*240/8=7200\text{B}$, 预计将会占用 $7200/256=28.125$ 个页, 因此存储这幅图片要占用 29 个页, 执行这条语句后 Demo1.bin 将会被写入到第 3600~3628 共计 29 个页中。

附录：图片生成.bin 文件的方法

MSU2 系列可编程 USB 屏幕目前仅支持 16 位彩色图片和单色图片的显示，下面分别讲解这两种图片的生成方法：

生成 16 位彩色图片的.bin 文件：

打开 Image2Lcd 2.9 软件，载入图像之后配置如下：



注意事项：

最终输出图像的宽度和高度并不一定是所设置的宽度和高度，在生成.bin 文件前应当确认上图红色圆圈中输出图像的宽度和高度是自己所需要的；在生成.bin 文件之前，建议将图片使用软件裁剪成需要的长宽比例之后再行转换。

生成单色图片的.bin 文件:

打开 Image2Lcd 2.9 软件，载入图像之后配置如下:



注意事项:

最终输出图像的宽度和高度并不一定是所设置的宽度和高度，在生成.bin 文件前应当确认上图红色圆圈中输出图像的宽度和高度是自己所需要的；在生成.bin 文件之前，建议将图片使用软件裁剪成需要的长宽比例之后再行转换。

其次，特别注意的是单色图像是 8 个像素点的数据合成 1B，而这款图像取模软件是每一行都用整数个 B 来存储数据，因此最好确保输出图像的宽度是 8 的整数倍。

②将 bin 格式字库文件写入闪存芯片(页地址, bin 格式字库文件名称)

Write_Flash_ZK(Page_add,ZK_name)

举例: Write_Flash_ZK(3651,'ASC64')#32*64 分辨率 ASCII 表格, 占用 128 个 Page

将 ASC64.bin 文件写入闪存芯片第 3651 页处(注意: 要写入的.bin 文件必须要和当下执行的 python 文件在同一个文件夹目录中; 当前页写不完.bin 文件时会自动写到下一页, 直到.bin 文件写完), ASC64.bin 是 32*64 分辨率的 ASCII 码字库文件, 按照 ASCII 码的顺序一共存储了 128 个字体; 每个字体可以看成是一张 32*64 分辨率的单色图片, 字库的大小为 $32*64*128/8=32768\text{B}$, 预计将会占用 $32768/256=128$ 个页, 因此存储这幅图片要占用 128 个页, 执行这条语句后 ASC64.bin 将会被写入到第 3651~3778 共计 128 个页中。

使用 32*64 分辨率的字体还有一个考量因素是单个字体占用的空间 $=32*64/8=256\text{B}$, 刚好为一个页的大小, 每个字体刚好占据一个完整的页, 在后续显示时可以直接调用单色图片显示的 API 接口; 当生成其他分辨率的字体时, 如果单个字体的大小并不是整数个页, 往往需要在主控固件准备相应分辨率的显示接口函数才行, 如后面专门给 16X16 汉字字体设置了专门的显示接口; 因此目前只能正确显示两种分辨率的字库文件, 一类是单个字体刚好占据整数个页(如: 32X64, 64*128, 96X192), 另一类是有专门显示接口的 16X16 字体;

附录：生成 bin 格式字库文件的方法

MSU2 系列可编程 USB 屏幕目前仅支持按 ASCII 码顺序排列的英文字库以及按照 GB2312 编码顺序的汉字库，下面分别讲解这两种字库的生成方法：

生成英文字库.bin 文件：

打开“字库生成器”软件，选择合适的字体之后配置如下：

点击“生成”后生成的文件需要自行添加上.bin 的后缀



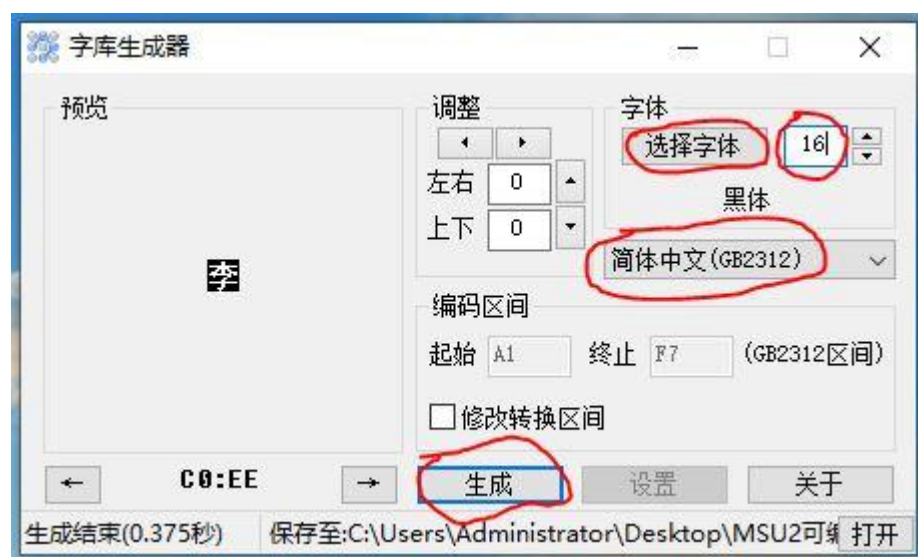
注意事项：

这款软件依赖些不太常用的.dll 文件，相当多的电脑系统是缺少这些.dll 文件的，进而导致软件无法运行，至于解决方法大家就自行查找；软件里面选择生成英文字库后，字体点阵是宽高比是 1:2，这就导致部分比较矮胖的字体是没法完全放到点阵中，建议应当选择高瘦一点的字体；

生成汉字库.bin 文件：

打开“字库生成器”软件，选择合适的字体之后配置如下：

点击“生成”后生成的文件需要自行添加上.bin 的后缀



注意事项：

这款软件依赖些不太常用的.dll 文件，相当多的电脑系统是缺少这些.dll 文件的，进而导致软件无法运行，至于解决方法大家就自行查找；软件里面选择生成汉字库后，字体点阵是宽高比是 1:1；建议选择 16X16 的，这是因为目前的主控固件只适配了 16X16 的字体，并且 16X16 的汉字库点阵需要消耗 255KB 的空间，已经占据了四分一的闪存芯片存储空间，过大的字体可能存储不下；

③显示一幅单色图片(图片的X轴的坐标,图片的Y轴坐标,图片的宽度,图片的高度,单色图片存储所在的页地址,图片的前景色(RGB565),图片的背景色(RGB565))

LCD_Photo_wb(LCD_X,LCD_Y,LCD_X_Size,LCD_Y_Size,Page_Add,LCD_FC,LCD_BC)

举例: LCD_Photo_wb(0,20,160,40,3600,0xf800,0x0000)

从闪存芯片 3600 页的位置读取一幅 160X80 分辨率的单色图片并将其显示到屏幕上,单色图片左上角坐标设置为(0,20),前景色(RGB565)为 0xf800(红色),背景色(RGB565)为 0x0000(黑色)

注意: 屏幕的 X,Y 坐标系和数学上 X,Y 坐标系并不一样,屏幕的 X,Y 坐标是以屏幕左上角为原点的;对于 160X80 分辨率的屏幕而言,X 的取值应当为 0~159, Y 的取值为 0~79;参数里面的图片的宽度和高度必须和所存储的单色图片的宽度和高度保持一致,否则将无法正确显示;图片的背景色和背景色应当设置为不同的颜色,否则将看不到需要显示的内容;

④显示一幅以指定彩色图片为背景的单色图片(图片的 X 轴的坐标,图片的 Y 轴坐标,图片的宽度,图片的高度,单色图片存储所在的页地址,图片的前景色(RGB565),彩色背景图片存储所在的页地址)

LCD_Photo_wb_MIX(LCD_X,LCD_Y,LCD_X_Size,LCD_Y_Size,Page_Add,LCD_FC,BG_Page):

举例: LCD_Photo_wb_MIX(0,20,32,64,3699,0xf800,3826)

从闪存芯片 3699 页的位置读取一幅 32X64 分辨率的单色图片并将其显示到屏幕上,单色图片左上角坐标设置为(0,20),前景色(RGB565)为 0xf800(红色),背景为存储在 3826 页位置的一幅 160X80 彩色图像;注意:屏幕的 X,Y 坐标系和数学上 X,Y 坐标系并不一样,屏幕的 X,Y 坐标是以屏幕左上角为原点的;对于 160X80 分辨率的屏幕而言,X 的取值应当为 0~159, Y 的取值为 0~79;参数里面的图片的宽度和高度必须和所存储的单色图片的宽度和高度保持一致,否则将无法正确显示;以及所使用的彩色背景图片的分辨率必须是和屏幕分辨率一致;